

ONEDRAFT

CAD-AI — Aria Powered

System Architecture Specification

Version 1.0

Architecture Status: Foundational Implementation Architecture

Product: OneDraft

AI System: Aria Powered

Primary Function: AI-assisted architectural design and coordinated documentation

Primary Output: Model-derived architectural document sets

1. ARCHITECTURAL OBJECTIVE

OneDraft shall be implemented as a modular, model-driven software platform in which artificial intelligence, parametric geometry, regulatory intelligence, document generation and customer interaction operate as coordinated services around a persistent Project Model.

The system shall not depend upon a single AI response to generate an architectural project.

Instead, OneDraft shall maintain a persistent computational state.

The fundamental architecture is:

USER

- APPLICATION
 - PROJECT ORCHESTRATOR
 - PROJECT MODEL
 - SPECIALIZED ENGINES
 - VALIDATION
 - DOCUMENT GENERATION
 - CUSTOMER REVIEW
 - REVISION
 - FINAL OUTPUT
-

2. SYSTEM LAYERS

OneDraft shall be divided into eight principal layers.

Layer 1 — Customer Experience

Layer 2 — Application/API Layer

Layer 3 — Aria Intelligence and Orchestration

Layer 4 — Project Model and Persistence

Layer 5 — Geometry / CAD / BIM Computation

Layer 6 — Regulatory and Validation Intelligence

Layer 7 — Rendering and Document Production

Layer 8 — Infrastructure / Security / Billing / Audit

Each layer shall have defined responsibilities.

No layer should unnecessarily become responsible for another layer's functions.

3. LAYER 1 — CUSTOMER EXPERIENCE

The customer interacts with OneDraft through a web-based application.

The primary interface shall be the:

OneDraft Studio

The Studio shall contain:

Project Dashboard

Project Specification Interface

AI Conversation Interface

Site Interface

Model Viewer

Floor/Level Navigator

Drawing Viewer

Redline Interface

Code/Compliance Panel

Revision History

Document Set Manager

Export Center

Account / Subscription

The interface shall progressively expose complexity according to the customer's role.

A homeowner should not be required to understand BIM terminology.

A professional architect or developer may access significantly deeper controls.

4. ROLE-BASED EXPERIENCE

OneDraft shall support role-based interfaces.

Initial roles:

Homeowner

Designer

Drafting Technician

Architect

Engineer

Developer

Project Manager

Enterprise Administrator

Reviewer

OneDraft Administrator

Each role receives permissions appropriate to its function.

For example, a homeowner may modify project requirements and submit redlines.

A professional user may additionally manipulate model parameters, drawing standards and documentation settings.

Enterprise users may administer multiple projects and organizations.

5. LAYER 2 — APPLICATION/API LAYER

The application layer provides the interface between the frontend and backend services.

Primary services include:

Authentication Service

Project Service

User Service

Organization Service

Specification Service

Model Service

Drawing Service

Revision Service

Redline Service

Code Service

Validation Service

Export Service

Billing Service

Notification Service

Audit Service

The frontend shall communicate with these services through controlled APIs.

6. API GATEWAY

OneDraft shall employ an API Gateway as the principal external entry point.

The gateway shall handle:

Authentication

Authorization

Rate Limiting

Request Validation

Service Routing

API Versioning

Logging

Security Controls

The client application should not directly access internal services that do not need public exposure.

7. LAYER 3 — ARIA INTELLIGENCE ENGINE

Aria Powered shall operate as a collection of coordinated AI capabilities rather than a single unrestricted conversational process.

The Aria layer shall include:

Intent Interpreter

Specification Parser

Architectural Reasoning Engine

Constraint Reasoner

Code Reasoning Interface

Spatial Reasoner

Design Generator

Revision Interpreter

Conflict Explanation Engine

Documentation Planner

Quality-Control Agent

Project Orchestrator

These components may use one or more underlying AI models.

The customer should experience them as one coherent intelligence.

8. ARIA ORCHESTRATOR

The Aria Orchestrator is the central intelligence coordinator.

It determines which operation is required.

For example:

Customer:

“Move the kitchen 1.5 metres east and maintain the existing exterior envelope.”

Aria Orchestrator determines:

Intent: Modify spatial configuration.

Then identifies:

Affected Space: Kitchen

Affected Geometry: Interior partitions

Constraint: Exterior envelope immutable

Potential Dependencies: Plumbing, electrical, cabinetry, windows, circulation

The orchestrator then submits the operation to the appropriate computational services.

Aria does not simply return prose.

It initiates a structured model operation.

9. STRUCTURED AI OPERATIONS

AI actions shall be represented as structured commands.

Conceptually:

CREATE_SPACE

MOVE_ELEMENT

ROTATE_ELEMENT

RESIZE_SPACE

ADD_LEVEL

CHANGE_ASSEMBLY

ADD_DOOR

ADD_WINDOW

MODIFY_REQUIREMENT

GENERATE_DRAWING

RUN_VALIDATION

CREATE_REDLINE

RESOLVE_REDLINE

REGENERATE_DOCUMENT_SET

The AI layer should therefore communicate with the Project Model through controlled operations rather than unrestricted database modification.

10. AI GUARDRAIL

Aria shall never directly manipulate production database records without passing through the model-command layer.

The operation becomes:

AI INTENT

↓

STRUCTURED COMMAND

↓

COMMAND VALIDATION

↓

PROJECT MODEL TRANSACTION

↓

GEOMETRY UPDATE

↓

VALIDATION

↓

COMMIT

This creates a transaction boundary around AI activity.

If validation fails, the transaction may be rejected or placed into a review state.

11. PROJECT ORCHESTRATOR

The Project Orchestrator manages multi-stage operations.

For example:

A customer changes the building footprint.

The orchestrator can determine that this requires:

Site impact analysis
Spatial regeneration
Structural coordination
Code validation
Dimension regeneration
Elevation regeneration
Section regeneration
Schedule updates
Drawing regeneration

The customer should experience this as one operation:

Updating Building Footprint...

rather than manually executing nine different processes.

12. LAYER 4 — PROJECT MODEL SERVICE

The Project Model Service owns the canonical project state.

It manages:

Project Objects

Relationships

Constraints

Versions

Dependencies

Model Transactions

Snapshots

Revision State

The service exposes controlled interfaces to other systems.

13. DATABASE ARCHITECTURE

OneDraft should use multiple persistence mechanisms rather than forcing every type of information into one database.

The architecture should distinguish:

Transactional Data

Geometric Data

Document Data

Search/Index Data

Object Storage

Audit Data

This allows each workload to use an appropriate storage mechanism.

14. TRANSACTIONAL DATABASE

A relational database should manage structured project information.

Core entities include:

Users

Organizations

Projects

Requirements

Levels

Spaces

Elements

Assemblies

Constraints

Revisions

Redlines

Drawings

Sheets

Code Rules

Code Manifests

Validation Results

Acceptance Records

Subscriptions

Invoices

Permissions

This database represents the structured project state.

15. OBJECT STORAGE

Large project artifacts shall be stored separately.

Examples:

PDFs

CAD files

BIM files

Rendered images

Uploaded drawings

Survey files

Reference photographs

Code source documents

Generated document packages

Project archives

Object storage records shall be associated with Project_ID and Version_ID.

16. MODEL VERSIONING

The project model shall support versioned state.

Conceptually:

PROJECT-001

MODEL-001

↓

MODEL-002

↓

MODEL-003

Every model revision must be reconstructable.

This allows OneDraft to answer:

What did the project look like before this redline?

and:

Which geometry produced this drawing?

17. EVENT LOG

Major project events shall generate immutable events.

Examples:

PROJECT_CREATED

SPECIFICATION_SUBMITTED

CODE_MANIFEST_CREATED

MODEL_GENERATED

ELEMENT_MODIFIED

VALIDATION_COMPLETED

REDLINE_CREATED

REDLINE_RESOLVED

DRAWINGS_GENERATED

CUSTOMER_ACCEPTED

DOCUMENT_EXPORTED

The event stream becomes a second layer of project provenance.

18. LAYER 5 — CAD GEOMETRY ENGINE

The Geometry Engine is responsible for converting model instructions into actual computational geometry.

It shall support:

2D CAD Geometry

3D Geometry

Parametric Geometry

Boolean Operations

Spatial Relationships

Coordinate Systems

Constraints

Assemblies

Views

Sections

Projections

Dimension Extraction

The engine is independent of Aria.

Aria instructs it.

The Geometry Engine performs the computation.

19. CAD REPRESENTATION

The internal model should maintain semantic objects rather than merely linework.

For example:

A wall should be:

Wall Object

rather than:

Four Lines

A door should be:

Door Object

rather than:

Two Arcs + Rectangle

A stair should be:

Stair Object

rather than:

Repeated Lines

This permits the system to understand what it is drawing.

20. PARAMETRIC MODEL

Elements shall possess parameters.

Example:

WALL-00427

Parameters:

Length

Height

Thickness

Assembly

Location

Fire Rating

Material

Acoustic Rating

Thermal Properties

Changing a parameter triggers dependency evaluation.

21. DEPENDENCY GRAPH

The Project Model shall maintain a dependency graph.

For example:

WALL - 00427

affects:

ROOM - 003

which affects:

DIM - 011

which affects:

PLAN - A121

and:

SECTION - A401

and:

ELEVATION-A301

The system can therefore determine the minimum set of outputs requiring regeneration.

This is a critical optimization.

One small change should not require blindly rebuilding the entire project.

22. DOCUMENTATION ENGINE

The Documentation Engine converts model information into architectural drawings.

Its responsibilities include:

View Creation

Linework Generation

Dimension Generation

Annotation

Tags

Schedules

Detail References

Section References

Sheet Composition

Title Blocks

Revision Blocks

Drawing Index

Graphic Standards

23. ARCH D OUTPUT

OneDraft shall support ARCH D sheet generation as a first-class document format.

The system should control:

Page Dimensions

Margins

Title Block

Drawing Scale

Line Weight

Text Hierarchy

Dimension Style

Graphic Symbols

North Arrow

Graphic Scale

Sheet Number

Revision Block

Project Information

The visual objective is professional architectural documentation rather than an AI-generated illustration.

24. RENDERING ENGINE

A separate Rendering Engine shall generate visual representations of the model.

Rendering modes may include:

Technical CAD

Blue-on-White Drawing

Axonometric

Architectural Visualization

Photorealistic

Conceptual

The renderer shall operate upon the same underlying model.

Thus:

CAD drawing

and:

architectural rendering

remain different representations of the same building.

25. SECTION ENGINE

Sections shall be computationally generated from the model.

The system identifies:

Section Plane

↓

Intersecting Geometry

↓

Cut Geometry

↓

Projection Geometry

↓

Annotations

↓

Dimensions

↓

Section Drawing

This prevents the common failure where a section visually resembles the building but does not correspond to its floor plans.

26. ELEVATION ENGINE

Elevations shall similarly be generated from model geometry.

An elevation shall be a projection of the actual building.

Changes to:

Windows

Doors

Roof Geometry

Wall Heights

Materials

Masses

should automatically affect the corresponding elevation.

27. SCHEDULE ENGINE

Schedules shall be generated from model objects.

Examples:

Door Schedule

Window Schedule

Room Schedule

Finish Schedule

Equipment Schedule

The schedule engine shall query the Project Model rather than rely on manually duplicated information.

28. LAYER 6 — REGULATORY INTELLIGENCE

The Code Intelligence Service shall operate separately from the architectural reasoning engine.

Its responsibilities include:

Jurisdiction Identification

Regulatory Source Identification

Version Identification

Effective-Date Determination

Rule Extraction

Applicability Analysis

Code Manifest Generation

Rule Verification

Change Detection

This separation is intentional.

Architectural reasoning should not silently invent regulatory requirements.

29. REGULATORY SOURCE PIPELINE

The regulatory pipeline is:

JURISDICTION

↓

SOURCE DISCOVERY

↓

SOURCE RETRIEVAL

↓

VERSION IDENTIFICATION

↓

DOCUMENT NORMALIZATION

↓

RULE EXTRACTION

↓

APPLICABILITY MODEL

↓

CODE MANIFEST

↓

PROJECT VALIDATION

The system should retain source provenance.

30. CODE HASHING / PROVENANCE

The regulatory baseline shall generate a cryptographic identifier.

Conceptually:

`HASH(Source_ID + Version + Effective_Date + Retrieved_Content +
Jurisdiction)`

The resulting manifest becomes associated with the project model.

If the regulatory dataset changes, the system creates a new manifest rather than silently replacing the old one.

31. VALIDATION ENGINE

The Validation Engine evaluates the project against:

Customer Requirements

Geometry

Spatial Rules

Code Rules

Accessibility

Egress

Life Safety

Zoning

Internal Coordination

Drawing Coordination

The engine returns machine-readable results.

Example:

RULE - 0047

STATUS: REVIEW_REQUIRED

AFFECTED ELEMENT: STAIR-002

REASON: Required condition requires professional verification.

This result can then appear in the user interface.

32. HUMAN REVIEW BOUNDARY

OneDraft shall distinguish between computational verification and professional certification.

The system may state:

“Model condition satisfies the configured computational rule.”

It should not automatically transform that statement into:

“Municipally approved.”

Similarly:

“Drawing generated.”

is different from:

“Drawing professionally sealed.”

and:

“Permit approved.”

The system maintains these states independently.

33. LAYER 7 — DOCUMENT PRODUCTION

The Document Production Service creates final deliverables.

The pipeline is:

PROJECT MODEL

↓

DRAWING VIEWS

↓

SHEET COMPOSITION

↓

QUALITY CONTROL

↓

DOCUMENT RENDERING

↓

PDF / CAD EXPORT

↓

PACKAGE CREATION

↓

HASH / VERSION

↓

34. AUTOMATED DRAWING QUALITY CONTROL

Before a document package becomes available for final acceptance, OneDraft should execute automated checks.

Checks include:

Missing Dimensions

Unresolved References

Duplicate Tags

Missing Sheet References

Broken Section References

Missing Room Tags

Missing Door/Window Tags

Drawing Scale

Title Block

Revision Consistency

Sheet Number Consistency

Model/Drawing Mismatch

Unresolved Redlines

Validation Failures

The result becomes a Document QA Report.

35. LAYER 8 — SECURITY

OneDraft shall treat project information as customer-controlled information.

Security architecture shall include:

Encrypted Transport

Encrypted Storage

Role-Based Access Control

Organization Isolation

Project-Level Authorization

Audit Logging

Session Security

API Authentication

Secrets Management

Backup

Recovery

Version Preservation

36. MULTI-TENANT ARCHITECTURE

OneDraft is intended to operate as a SaaS platform.

The logical hierarchy is:

OneDraft Platform

→ **Organization**

→ **Users**

→ **Projects**

→ **Models**

→ **Documents**

Each project must be isolated from unrelated organizations.

Enterprise customers may require additional tenant isolation.

37. SUBSCRIPTION ENGINE

The Billing Service controls access to computational resources.

Subscription tiers may eventually differentiate:

Projects

AI Usage

Model Complexity

Document Generation

Storage

Rendering

Collaboration

API Access

Enterprise Features

The billing system should never alter project data.

It controls entitlement.

38. JOB QUEUE

Long-running computational tasks shall operate asynchronously.

Examples:

Generate Building Model

Generate 100+ Drawing Sheets

Render High-Resolution Images

Run Full Code Validation

Generate Document Package

Rebuild Affected Views

These operations enter a job queue.

The interface can display:

QUEUED

→ **PROCESSING**

→ **VALIDATING**

→ **COMPLETED**

or:

FAILED

with a diagnostic record.

39. COMPUTATIONAL WORKERS

Worker services execute queued operations.

Potential worker classes:

AI Worker

Geometry Worker

Validation Worker

Rendering Worker

Document Worker

Code Worker

Workers should be horizontally scalable.

A large commercial project may require substantially more computational resources than a small residential project.

40. API BETWEEN ARIA AND THE MODEL

The critical interface is:

ARIA

↑

MODEL COMMAND API

The command API becomes the controlled boundary between intelligence and project state.

Example conceptual operation:

MODIFY_SPACE

Parameters:

Space_ID

Target

Constraint

Requested Change

Preserve

Dependencies

The system then validates the operation before committing it.

41. API BETWEEN MODEL AND GEOMETRY

The Geometry API translates semantic model changes into geometric computation.

For example:

MODEL

Wall W-427 moves 600 mm east.

↓

GEOMETRY ENGINE

Calculate new wall geometry.

↓

Update affected rooms.

↓

Recalculate dependent dimensions.

↓

Recalculate intersections.

↓

Generate affected views.

42. API BETWEEN MODEL AND DOCUMENTATION

The Documentation API asks:

What representation of this model is required?

The answer may be:

Floor Plan A121

Section A401

Elevation A301

Door Schedule A601

Each view is generated from model state.

43. API BETWEEN MODEL AND VALIDATION

The Validation API receives:

Project State

-

Applicable Rules

and returns:

Validation Results

These results become persistent project objects.

44. CUSTOMER REDLINE LOOP

The complete operational loop becomes:

CUSTOMER

↓

Redline

↓

ARIA INTERPRETS REDLINE

↓

MODEL COMMAND

↓

GEOMETRY UPDATE

↓

DEPENDENCY GRAPH

↓

VALIDATION

↓

DRAWING REGENERATION

↓

DOCUMENT QA

↓

CUSTOMER REVIEW

The loop continues until the customer accepts the revision.

45. FINAL DOCUMENT EVENT

When the customer accepts a revision, OneDraft creates a Finalization Event.

The event captures:

Project_ID

Model Version

Drawing Set

Revision

Code Manifest

Validation Report

Redline Resolution

Document Hashes

Customer Acceptance

Timestamp

This creates a reproducible project record.

46. APPLICATION DEPLOYMENT

The initial production architecture should be cloud-native.

Conceptually:

Web Client

↓

API Gateway

↓

Application Services

↓

Message Queue

↓

Computational Workers

↓

Project Database

Object Storage

Search Index

Audit Store

The architecture should permit individual computational services to scale independently.

47. DEVELOPMENT STRATEGY

The system should not attempt to implement every discipline simultaneously.

The first functional product should establish:

Project Creation

Specification Input

Aria Interpretation

Project Model

Basic Parametric Geometry

Floor Plans

Elevations

Sections

ARCH D Sheets

Redlines

Revision

PDF Export

Once that loop works reliably, progressively add:

Advanced Code Intelligence

Structural Coordination

MEP Coordination

BIM Interoperability

Enterprise Collaboration

Professional Review Workflows

Advanced Rendering

Developer-scale automation

48. THE FIRST MVP

The minimum viable OneDraft system should therefore accomplish one complete loop:

A user describes a building → OneDraft constructs a structured project → CAD-AI creates the model → drawings are generated → the user redlines the project → CAD-AI modifies the model → affected drawings regenerate → the user accepts the final revision → OneDraft produces the final document package.

If this loop works reliably, the fundamental product exists.

Everything else becomes expansion.

49. DEFINING DIFFERENTIATOR

OneDraft's principal technological differentiator is not simply that Aria can produce architectural drawings.

It is that:

THE AI, PROJECT MODEL, GEOMETRY, REGULATORY CONTROLS, DRAWINGS, REDLINES AND FINAL DOCUMENTATION ALL PARTICIPATE IN ONE VERSIONED SYSTEM.

That creates continuity between:

What the customer requested

What the AI interpreted

What the model contains

What the geometry represents

What the code engine evaluated

What the drawings depict

What changed

What was accepted

and:

What was ultimately exported.

50. SYSTEM ARCHITECTURE — MASTER FLOW

The complete OneDraft system is therefore:

CUSTOMER

↓

ONEDRAFT STUDIO

↓

API GATEWAY

↓

PROJECT ORCHESTRATOR

↙ ↓ ↘

ARIA | PROJECT MODEL | CODE INTELLIGENCE

↓ ↓ ↓

AI COMMANDS | PARAMETRIC STATE | CODE MANIFEST

↘ ↓ ↙

GEOMETRY ENGINE

↓

DEPENDENCY GRAPH

↓

VALIDATION ENGINE

↓

DOCUMENTATION ENGINE

↓

DRAWING / RENDERING ENGINE

↓

DOCUMENT QA

↓

REDLINE / REVISION ENGINE

↓

CUSTOMER ACCEPTANCE

↓

FINAL DOCUMENT PACKAGE

↓

VERSIONED ARCHIVE

51. ARCHITECTURAL PRINCIPLE

OneDraft is consequently not architected as:

ChatGPT + CAD

It is architected as:

A persistent architectural project system with Aria functioning as its intelligent reasoning and orchestration layer.

The AI becomes the interface through which the customer can communicate with the project model.

The project model becomes the persistent source of truth.

The geometry engine becomes the computational drafting mechanism.

The regulatory engine becomes the jurisdictional control mechanism.

The validation engine becomes the consistency mechanism.

The documentation engine becomes the professional drawing-production mechanism.

The revision system becomes the collaborative mechanism.

And the provenance layer becomes the historical record of how the final project was produced.

Version 0.1 — OneDraft System Architecture established.